

The Perceptron Trick in Code

Contents

1. Overview	2
2. The data	2
3. The code	3
4. Drawing the line	3
5. Watching the line move	4
6. The weakness: the line stops too early	5
6.1 Comparing with logistic regression	5
6.2 Why it matters	6
7. Summary	7
8. Key terms	7

Where this fits: Pipeline map step 8 (Model). See the [Course map](#).



- **Builds on:** Classification problems ([Note 3](#)); Model-based learning ([Note 6](#)); Gradient descent ([Note 57](#)); Polynomial features ([Note 61](#)); Equation of a hyperplane ([Note 70](#)).
- **Leads to:** Softmax regression ([Note 79](#)); Support vector machines ([Note 92](#)).
- **Compare with:** Support vector machines ([Note 92](#)).

1. Overview

Key point: The perceptron trick takes about ten lines of Python. It finds a line that separates the classes, but it stops as soon as no point is misclassified, so the line can end up very close to one class. Logistic regression keeps improving until the line sits well between them.

The previous Note described the perceptron trick: start with any line, pick random points, and move the line towards each misclassified one. This Note codes it, watches the line move, and compares the result with scikit-learn's `LogisticRegression`. The comparison shows the trick's weakness, which the following Notes fix.

2. The data

Key point: 100 points with two inputs and two balanced classes, made with `make_classification`.

`make_classification` creates classification data, just as `make_regression` creates regression data.

Python: Creating the data.

```
from sklearn.datasets import make_classification
```

```
X, y = make_classification(n_samples=100, n_features=2,  
    n_informative=1, n_redundant=0, n_classes=2,  
    n_clusters_per_class=1, random_state=41,  
    hypercube=False, class_sep=10)
```

`X` has 100 rows and 2 columns (x_1 and x_2), and `y` holds the class of each row: 50 zeros and 50 ones. `class_sep` sets how far apart the classes are. In the figures, class 1 is green and class 0 is blue.

3. The code

Key point: Add a column of 1s, start with weights of 1, and repeat 1,000 times: pick a random row, predict it with the step function, and apply $w = w + \text{lr} (y - \hat{y}) x$.

Python: The perceptron trick.

```
def step(z):
    return 1 if z > 0 else 0

def perceptron(X, y, lr=0.1, loops=1000, seed=0):
    rng = np.random.default_rng(seed)
    X = np.insert(X, 0, 1, axis=1)      # column of 1s
    weights = np.ones(X.shape[1])      # [w0, w1, w2]
    for _ in range(loops):
        j = rng.integers(0, X.shape[0]) # random row
        y_hat = step(X[j] @ weights)    # predict it
        weights = weights + lr * (y[j] - y_hat) * X[j]
    return weights[0], weights[1:]

intercept_, coef_ = perceptron(X, y)
```

Line by line:

- `np.insert(X, 0, 1, axis=1)` puts a 1 at the start of every row, so each row becomes $(1, x_1, x_2)$. The first weight then acts as the intercept.
- `weights` starts as $(1, 1, 1)$: the line $1 + x_1 + x_2 = 0$.
- In each loop, `X[j] @ weights` is the dot product $w_0 + w_1x_1 + w_2x_2$ for row j , and `step` turns it into a prediction of 1 or 0.
- The update changes the weights only when $y \neq \hat{y}$ (the previous Note's table).

With seed 0, the result is $w_0 = 1.00$, $w_1 = 1.34$, $w_2 = 0.19$, and no training point is misclassified.

Extra: The fixed seed makes the random picks the same on every run, so the results repeat. Without it, every run gives a slightly different line.

4. Drawing the line

Key point: To plot $w_0 + w_1x_1 + w_2x_2 = 0$, solve for x_2 : slope $m = -w_1/w_2$, intercept $b = -w_0/w_2$.

To draw the line as $x_2 = mx_1 + b$, solve $w_0 + w_1x_1 + w_2x_2 = 0$ for x_2 :

$$x_2 = -\frac{w_1}{w_2}x_1 - \frac{w_0}{w_2}$$

With the weights above: $m = -1.34/0.19 = -7.02$ and $b = -1.00/0.19 = -5.23$. The line is

steep because w_2 is small: the classes are separated mainly by x_1 .

5. Watching the line move

Key point: The line moved only 6 times in 1,000 loops. After loop 108 every point was on its correct side, so nothing changed again.

Figure 1 animates the run. A red ring marks each picked point that was misclassified, and the line jumps towards it.

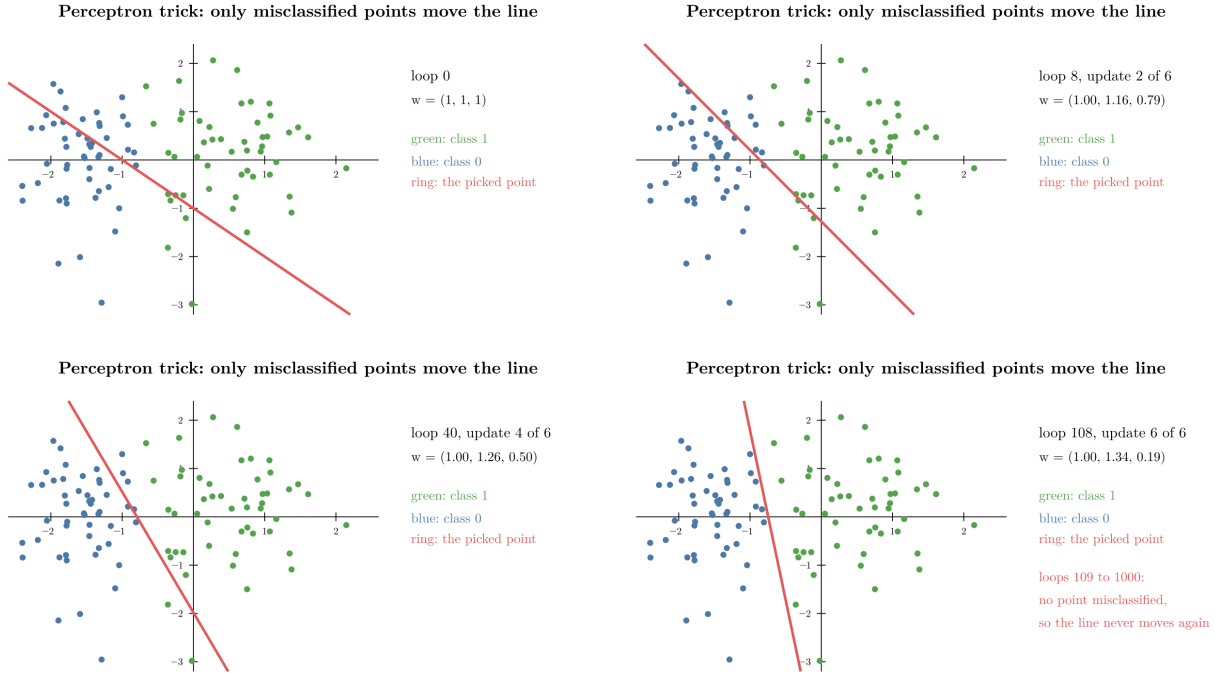


Figure 1: The perceptron trick on 100 points: the line after 0, 2, 4 and 6 updates

Update	Loop	Picked point's class	Weights (w_0, w_1, w_2) after
start	0		(1, 1, 1)
1	1	0	(0.90, 1.19, 0.86)
2	8	1	(1.00, 1.16, 0.79)
3	31	1	(1.10, 1.13, 0.60)
4	40	0	(1.00, 1.26, 0.50)
5	44	1	(1.10, 1.26, 0.21)
6	108	0	(1.00, 1.34, 0.19)

Most loops pick a point that is already correctly classified, and the line stays still. Only the occasional misclassified point pulls it. After update 6 there are no misclassified points left, so loops 109 to 1,000 change nothing.

6. The weakness: the line stops too early

Key point: Five random orders give five different lines, some almost touching one class. Logistic regression places its line with a fair gap to both classes.

6.1 Comparing with logistic regression

Key point: Fit scikit-learn's LogisticRegression on the same data and draw both lines.

Python: Logistic regression in scikit-learn.

```
from sklearn.linear_model import LogisticRegression
```

```
lor = LogisticRegression(C=100)
lor.fit(X, y)
m = -lor.coef_[0][0] / lor.coef_[0][1]
b = -lor.intercept_[0] / lor.coef_[0][1]
```

Figure 2 draws the perceptron lines from five random orders (seeds 0 to 4) in red and the logistic regression line in black.

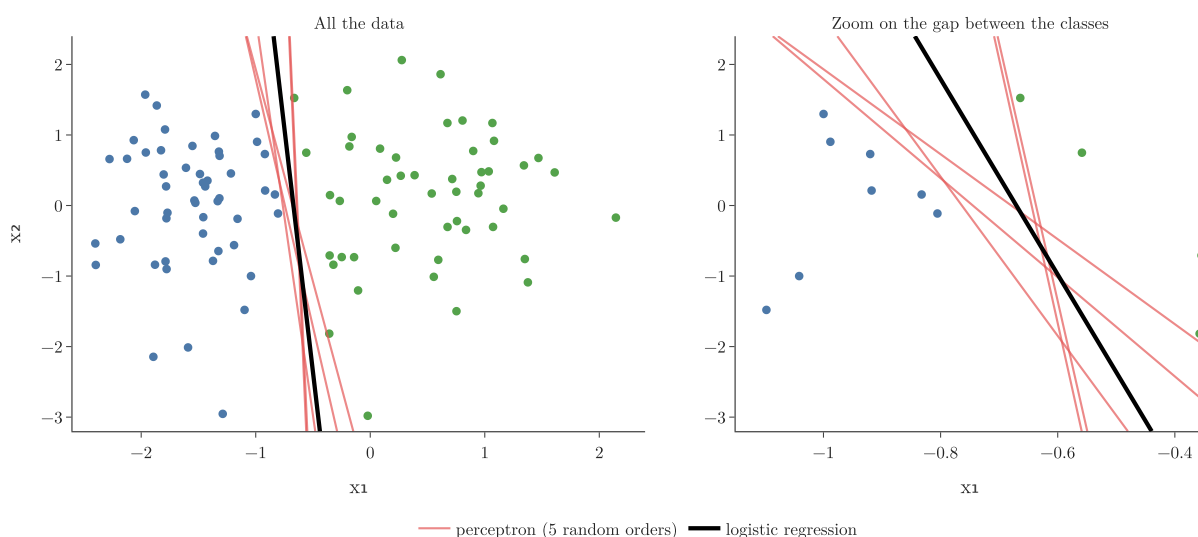


Figure 2: Five perceptron lines and the logistic regression line, with a zoom on the gap

All six lines separate the training data perfectly. The difference is where they sit in the gap between the classes. The table gives the distance from each line to the nearest point of each class:

Model	Gap to class 1	Gap to class 0
Perceptron, seed 0	0.127	0.065

Model	Gap to class 1	Gap to class 0
Perceptron, seed 1	0.234	0.052
Perceptron, seed 2	0.021	0.164
Perceptron, seed 3	0.020	0.104
Perceptron, seed 4	0.015	0.171
Logistic regression	0.117	0.143

Three of the five perceptron lines pass within about 0.02 of a green point. The logistic regression line keeps a similar distance from both classes.

6.2 Why it matters

Key point: A line that hugs one class misclassifies new points from that class easily. Zero training error is not the same as a good model.

The perceptron trick stops improving as soon as the training error is zero: its only goal is “no misclassified points”. Where exactly the line ends up depends on the random order of picks.

A new student whose point falls just next to the green cluster could easily land on the wrong side of a line that hugs it. A line in the middle of the gap leaves room for such points. So the perceptron can reach zero training error and still do worse on new data: it does not generalise as well.

Logistic regression keeps adjusting the line even when every training point is already correct, until the line is placed as well as possible. How it decides what “as well as possible” means is the subject of the next Notes: the sigmoid function, and then the loss function.

Extra: `C=100` makes scikit-learn’s regularisation weak. With the default `C=1`, `LogisticRegression` adds a Ridge-like penalty that keeps the weights small, which on this data moves its line slightly (it then misclassifies one point by a hair). The hyperparameters of logistic regression get their own Note later.

7. Summary

- The perceptron trick in code: a column of 1s, weights of 1, and 1,000 loops of $w \leftarrow w + \eta(y - \hat{y})x$.
- The line is drawn with slope $-w_1/w_2$ and intercept $-w_0/w_2$.
- On the example, the line moved only 6 times; after that, no point was misclassified and it stopped.
- Its final position depends on the random order and can hug one class.
- Logistic regression places the line with a fair gap to both classes, which generalises better.

8. Key terms

Term	Meaning
<code>make_classification</code>	scikit-learn function that creates random classification data
<code>class_sep</code>	<code>make_classification</code> setting for how far apart the classes are
Generalisation	How well a model performs on new data it was not trained on
Margin (gap)	The distance from a separating line to the nearest point of a class